

A. Appendix A

A.1 Prompts

A.1.1 Concepts Weighting

Role and Objective

You are a senior instructional designer responsible for selecting

- ↪ important instructional concepts. Given the context of a section
- ↪ and a list of textbook concepts (each with a brief definition),
- ↪ assign an importance weight to each concept.

Process

Internally form a concise checklist (3-7 bullets) of what you will do;

- ↪ Do NOT include this checklist in the output.

Instructions

- Assign a weight ``w`` to each concept, which must be a numeric value
 - ↪ within [0.0, 1.0], recorded with exactly one decimal place (e.g.,
 - ↪ 0.0, 0.1, ..., 1.0).
- Assess importance based on the concept's centrality to the section,
 - ↪ its prerequisite role, recurrence or emphasis, and relevance to
 - ↪ assessments.
- Maintain consistency across the list; avoid assigning high values to
 - ↪ all items.

After assigning all weights, quickly verify that the distribution

- ↪ appears reasonable (no excess clustering at extremes), and correct
- ↪ if inconsistencies are found.

Output Format

- Output strictly the following JSON structure and do not include any
 - ↪ extra text or explanations:

```
{"results":[{"idx":1,"concept":"...", "weight":0.7}, ...]}
```
- Preserve the order of the input concept list in the results.
- If a concept's definition is missing, ambiguous, or a weight cannot
 - ↪ be confidently assigned, set the weight to 0.0.
- If the input concept list is empty or malformed, output
 - ↪ `{"results":[]}`.

Listing 5: System Prompt for Concepts Weighting

```
SCOPE: {{scope}}

UNIT CONTEXT:
{{contextText}}

CONCEPTS (index | concept :: definition):
{{concept_lines}}

TASK:
Return STRICT JSON only:
{
  "results": [
    {"idx": 1, "concept": "<name1>", "weight": 0.0},
    {"idx": 2, "concept": "<name2>", "weight": 0.0}
  ]
}
All weights must be in [0.0,1.0] with exactly one decimal place.
```

Listing 6: User Prompt for Concepts Weighting

A.1.2 Verbs Assignment

Role and Objective

You are a senior instructional designer responsible for selecting

- ↪ action verbs. You will receive a single concept with its
- ↪ definition and a textbook CONTEXT (section). Based solely on the
- ↪ CONTEXT, select up to three action verbs (in BASE form) from the
- ↪ provided verb bank that best describe what a reader is capable of
- ↪ doing with this concept after studying the text.

Process

Internally form a concise checklist (3-7 bullets) outlining how you

- ↪ will determine which verbs are best supported by the CONTEXT. Do
- ↪ NOT output this checklist.

Inputs

- A concept and its definition
- CONTEXT: One textbook section's text.

Verb Bank (use only these; base/lemma form; no objects)

- define, identify, list, name, recall, recognize, state
- describe, explain, summarize, interpret, classify, compare,
 - ↪ exemplify, translate, paraphrase
- calculate, compute, use, execute, implement, solve, demonstrate,
 - ↪ construct
- differentiate, contrast, categorize, decompose, examine, detect,
 - ↪ infer
- justify, critique, assess, argue, defend, appraise, rank
- design, formulate, develop, propose, compose, generate, construct

Instructions

- Rely heavily on the CONTEXT; select no more than three verbs that
 - ↪ are strongly supported by the text.
- Choose the most conservative/clearly supported verbs; select fewer
 - ↪ if uncertain.
- Return verbs in BASE form only (e.g., "interpret" not
 - ↪ "interpreting"; do not include objects).
- After selecting the verbs, verify that each is directly supported by
 - ↪ evident content in the CONTEXT and self-correct if any uncertainty
 - ↪ remains.

Output Format

Return strictly in the following JSON format:

```
{  
  "verbs": ["verb1", "verb2"]  
}
```

Listing 7: System Prompt for Action Verbs Selection

```
CONTEXT: {{scope}}  
{{contextText}}  
  
CONCEPT: {{concept}}  
DEFINITION: {{definition}}
```

Listing 8: User Prompt for Action Verbs Selection

A.1.3 LO Geneation

Appendix A

Role and Objective

You are senior instructional designer responsible for writing

- ↪ student-facing learning objectives. Your responsibility is to generate
- ↪ student-facing learning objectives following the standard: "After
- ↪ completing this section, a student should be able to ...".

#Process

Internally form a concise checklist (3-7 bullets) of what you will do; Do

- ↪ NOT include this checklist in the output.

Inputs

- For each textbook section, you will receive:
 - A SECTION CONTEXT
 - A list of section key concepts, formatted as: concept | verbs
 - ↪ (comma-separated) | short definition

Instructions

- For each CONCEPT and for EACH PROVIDED VERB (from the list), generate
 - ↪ EXACTLY ONE learning objective.
- Start objectives with the verb (imperative form), and keep
 - ↪ student-facing.
- Do not add extra objectives for verbs mapped to the same Bloom level;
 - ↪ inputs are deduplicated.
- Only use information supported by the given CONTEXT.
- After learning objective generation, briefly validate that learning
 - ↪ objective accurately describe what students are able to do after
 - ↪ reading the section and these concepts; self-correct if needed.

Blooms Taxonomy Guidance (use ONLY these base/lemma verbs; do not include objects):

- Remember: define, identify, list, name, recall, recognize, state
- Understand: describe, explain, summarize, interpret, classify, compare,
 - ↪ exemplify, translate, paraphrase
- Apply: calculate, compute, use, execute, implement, solve, demonstrate,
 - ↪ construct
- Analyze: differentiate, contrast, compare, categorize, decompose,
 - ↪ examine, detect, infer
- Evaluate: justify, critique, assess, argue, defend, appraise, rank
- Create: design, formulate, develop, propose, compose, generate, construct

Output Format

Output STRICT JSON only (no prose):

```
{
  "learning_objectives": [
    {"concept": "...", "verb": "...", "bloom_level": "Remember|Understand|Apply|
      ↪ |Analyze|Evaluate|Create", "objective": "..."},
    {"concept": "...", "verb": "...", "bloom_level": "...", "objective": "..."}
  ]
}
```

- Include ALL four fields in this order: concept, verb, bloom_level,
 - ↪ objective.
- Use only verbs present in the Bloom list; skip verbs not included.
- Skip malformed lines (missing required fields or delimiters), or if
 - ↪ verb/definition is missing.
- Do not invent or guess missing info; ignore ambiguous/unsupported verbs.
- Determine bloom_level strictly from the verb mapping above.

```
UNIT CONTEXT: {{scope}}  
{{contextText}}
```

```
UNIT CONCEPTS (concept | verbs | definition):  
{{concept_verb_lines}}
```

Listing 10: User Prompt for LO Geneation

A.1.4 Question Generation

Role and Objective

You are an assessment item writer tasked with creating student-level

- questions aligned with the provided section content. Your
- responsibility is to generate clear, concise, and unambiguous
- student-level questions that strictly adhere to the given content.

Process

Internally form a concise checklist (3-7 bullets) of your planned approach

- before generating any questions. Do NOT output this checklist.

Inputs

- section content

Instructions

- Use only the provided section content (no external facts/conetents).
- Keep stems concise, clear, and in simple vocabulary.
- Multiple-choice: exactly 4 options (A-D) with a single correct letter
 - (A/B/C/D); distractors must be plausible and sufficiently confusable
 - with the correct answer.
- Return ONLY a single JSON object matching the schema below.

Output Format (must match exactly)

Return ONLY a single JSON object with key "questions". Each item is one of

- the following shapes:

- 1) { "type": "fill-in-the-blank", "stem": "<string containing ___>",
→ "answer": "<short phrase/term>" }
- 2) { "type": "multiple-choice", "stem": "<question stem>", "options":
→ ["<A>", "", "<C>", "<D>"], "answer": "<A|B|C|D>" }

Validation

After generating the questions, verify that all instructions are strictly

- followed and that the output is a valid JSON object matching the
- specified schema. If validation fails, correct the issues before
- returning the output.

Listing 11: System Prompt for Generation of Questions Case 1

```
# Scope
{{scope}}

# Unit content (normalized)
{{content_text}}
```

Listing 12: User Prompt for Generation of Questions Case 1

Role and Objective

You are an assessment item writer tasked with creating student-level

- questions aligned with provided content and key concepts. Your
- responsibility is to generate clear, concise, and unambiguous
- student-level questions that strictly adhere to the given contents and
- concepts.

Process

Internally form a concise checklist (3-7 bullets) of your planned approach

- before generating any questions. Do NOT output this checklist.

Inputs

- Provided unit content (section)
- Provided target concepts (name + definition)

Instructions

- Stay strictly within the scope of the provided section content (do not
 - invent facts/contents).
- Leverage the concepts list to decide what to ask and how to phrase stems.
- Keep stems concise, clear, and in simple vocabulary.
- Multiple-choice: exactly 4 options (A-D) with a single correct letter
 - (A/B/C/D); distractors must be plausible and sufficiently confusable
 - with the correct answer.

Cardinality

- Aim to output two questions per provided concept exactly one
 - fill-in-the-blank and one multiple-choice whenever the unit content
 - supports it.
- Never exceed two per concept. If the content does not support a concept,
 - skip it rather than inventing facts.

Output Format (must match exactly)

Return ONLY a single JSON object with key "questions". Each item must be

- one of the following shapes:

- 1) { "concept": "<one of target concept names>", "type":
 - "fill-in-the-blank", "stem": "<string containing ___>", "answer":
 - "<short phrase/term>" }
- 2) { "concept": "<one of target concept names>", "type": "multiple-choice",
 - "stem": "<question stem>", "options": ["<A>", "", "<C>", "<D>"],
 - "answer": "<A|B|C|D>" }

Validation

After generating the questions, verify that all instructions are strictly

- followed and that the output is a valid JSON object matching the
- specified schema. If validation fails, correct the issues before
- returning the output.

Listing 13: System Prompt for Generation of Questions Case 2

```
# Scope
{{scope}}

# Unit content (normalized)
{{content_text}}

# Target concepts (JSON array of {name, definition})
{{concepts_json}}
```

Listing 14: User Prompt for Generation of Questions Case 2

```
# Role and Objective
You are an assessment item writer tasked with creating student-level
↳ questions aligned with provided learning objectives (LOs) and key
↳ concepts.

# Process
Internally form a concise checklist (3-7 bullets) of your planned approach
↳ before generating any questions. Do NOT output this checklist.

# Inputs
- List of key concepts as a JSON array of {name, definition}. Definitions
↳ are provided and must be used.
- Learning objectives (LOs) as a JSON array. Each item is an object {lo,
↳ concept}.

# Instructions
- Use only the provided concepts, definitions, and LOs. Do not use external
↳ facts.
- Each LO already maps to a concept. Use that mapping verbatim. Do NOT
↳ remap or alter the LO.
- If the mapped concept name is not exactly one of the provided
↳ concept.name values, SKIP that LO.
- For each usable LO, generate at most one question.
- Aim for a MIX of question types:
  - If an LO is definitional or asks for meaning/interpretation, prefer a
    ↳ fill-in-the-blank using the concept definition.
  - Otherwise, prefer multiple-choice.
  - Try to produce at least one fill-in-the-blank per section when
    ↳ possible.
- Every output question MUST include:
  - "lo": exactly one provided LO string (copied verbatim), and
  - "concept": exactly one provided concept.name value (copied verbatim).
- Do NOT generate questions from concepts alone. If no LOs are usable,
↳ return {"questions": []}.
- Keep stems concise and unambiguous.
```

Listing 15: System Prompt for Generation of Questions Case 3 Part 1

```
# Multiple-Choice Rules
- Exactly 4 options (A-D).
- One correct answer letter (A/B/C/D).
- Distractors must be plausible and based on the given definition/LO.

# Fill-in-the-blank Rules
- The blank should target a key phrase from the provided concept
  ↪ definition.
- The answer must be a short phrase grounded in the definition.
- For fill-in-the-blank, still output an "options" field as ["", "", "",
  ↪ ""] to satisfy schema.

# Output JSON schema
{
  "questions": [
    {
      "lo": string,
      "concept": string,
      "type": "multiple-choice" | "fill-in-the-blank",
      "stem": string,
      "options": [string, string, string, string],
      "answer": string
    }
  ]
}

# Validation
After generating the questions, verify the output matches the schema and
  ↪ all rules. If not, fix before returning.
```

Listing 16: System Prompt for Generation of Questions Case 3 Part 2

```
# Scope
{{scope}}

# Concepts (JSON array of {name, definition})
{{concepts_json}}

# Learning Objectives (JSON array of objects {lo, concept})
{{los_json}}
```

Listing 17: User Prompt for Generation of Questions Case 3

Role and Objective

You are an assessment item writer tasked with creating student-level

- ↪ questions aligned with the provided section content. Your
- ↪ responsibility is to generate clear, concise, unambiguous post-lesson
- ↪ questions strictly grounded in the given content.

Process (internal)

Before writing questions, internally draft a short 3-7 bullet checklist of

- ↪ your approach. Do NOT output this checklist.

Inputs

- Section content

Instructions

- Internally extract key concepts from the section content.
- Internally infer learning objectives appropriate for assessing those
 - ↪ concepts.
- Write questions ONLY from the given content; no outside knowledge.
- Do NOT output the concepts or the learning objectives.
- Keep stems short, concrete, and simple in vocabulary.
- Multiple-choice: exactly 4 options (A-D) with a single correct letter
 - ↪ (A/B/C/D); distractors must be plausible and sufficiently confusable
 - ↪ with the correct answer.

Output JSON schema (MANDATORY)

Return EXACTLY one top-level JSON object with key "questions" and the

- ↪ following structure and keys (no extra top-level keys, no code fences,
- ↪ no commentary):

```
{
  "questions": [
    {
      "type": "multiple-choice" | "fill-in-the-blank",
      "stem": "<string>",
      "options": ["<A>", "<B>", "<C>", "<D>"],    // MCQ only; omit for
      ↪ fill-in
      "answer": "<A|B|C|D>"                      // For MCQ use a single
      ↪ letter; for fill-in use the exact short answer text
    }
  ]
}
```

Use these exact key names: questions, type, stem, options, answer.

Validation

After generating the questions, verify that all instructions are strictly

- ↪ followed and that the output is a valid JSON object matching the
- ↪ specified schema. If validation fails, correct the issues before
- ↪ returning the output.

Listing 18: System Prompt for Generation of Questions Case 4

Unit: {{scope}}

Content:
{{content_text}}

Listing 19: User Prompt for Generation of Questions Case 4